COP290 — Assignment 3

LevelDB: Range Scan, Range Delete, and Manual Full Compaction

Design Practices in Computer Science

Course	COP290 — Design Practices
Assignment	Assignment 3 — LevelDB
GitHub Repository	https://github.com/Mayank0909963/leveldb

Abstract

This report describes the design and implementation of three new APIs added to the LevelDB key-value storage engine: Scan(start_key, end_key) for ordered range reads, DeleteRange(start_key, end_key) for bulk deletions, and ForceFullCompaction() for synchronous full compaction with statistics reporting. The implementation reuses LevelDB's existing iterator, write-batch, and manual-compaction infrastructure, ensuring tight integration with the LSM-tree storage engine. Each API is grounded in a concrete analysis of LevelDB's write path, read path, and compaction mechanism, with explicit references to the source files (db/db_impl.cc, db/db_impl.h, include/leveldb/db.h, db/memtable.cc, db/version_set.cc, table/merger.cc, db/db_iter.cc). Correctness is validated by a model-based randomized test suite (937 / 937 checks passed) that compares the database against a reference std::map under randomized Put/Delete/DeleteRange/Scan workloads, with periodic forced compactions to exercise both MemTable and SSTable paths.

Contents

1 Analysis of LevelDB Write Path, Read Path, and Compaction

1.1 Write Path

1.2 Read Path

1.3 Iterator-Based Access

1.4 Compaction Mechanism

2 Range Scan — Design and Implementation

3 Range Delete — Design and Implementation

4 Manual Full Compaction — Design and Implementation

5 Evaluation

6 Source Code Reference Map

7 Conclusion

1 Analysis of LevelDB Write Path, Read Path, and Compaction

LevelDB is an embedded key-value store implemented as a Log-Structured Merge (LSM) tree. All client traffic enters through the DB interface declared in include/leveldb/db.h and is served by the DBImpl class in db/db_impl.{h,cc}. The storage engine has three layers: an in-memory MemTable (a skip list backed by an Arena), an immutable MemTable being flushed, and SSTables organised into seven on-disk levels (config::kNumLevels = 7). All three new APIs sit on top of this stack and inherit its atomicity, durability, and snapshot semantics.

1.1 Write Path

A call to DB::Put(options, key, value) is forwarded by the wrapper at db/db_impl.cc:1276 to DB::Put which packages the (key, value) pair into a WriteBatch and invokes DBImpl::Write(). Internally, the write traverses five well-defined stages.

Stage 1 — Writer queue and group commit (db/db_impl.cc:1284–1355).

Each caller wraps its WriteBatch in a Writer and pushes it onto the deque writers_. Only the writer at the front of the queue makes progress; others sleep on a condition variable (Writer::cv). The leader calls BuildBatchGroup() (db/db_impl.cc:1359) to coalesce pending WriteBatches into a single merged batch capped by max_size, amortising lock acquisition and I/O across many concurrent writes.

Stage 2 — Write-Ahead Log (db/db_impl.cc:1314–1318).

Before mutating in-memory state, the merged batch is serialised as a single record and appended to the WAL via log_->AddRecord(). The log writer (db/log_writer.cc) emits 32 KiB physical blocks. If WriteOptions::sync is set, the log file is fsynced; otherwise durability is best-effort and crash recovery replays the log on next open.

Stage 3 — MemTable insertion (db/db_impl.cc:1323).

WriteBatchInternal::InsertInto() walks the merged batch and applies each record to the active MemTable mem_. The MemTable is a lock-free skip list (db/skiplist.h, db/memtable.cc:22) keyed by the internal key user_key | seq#(56) | type(8); the value type distinguishes kTypeValue from kTypeDeletion, enabling tombstones to flow through the same code path as live writes (db/memtable.cc:76–99). The sequence number is the engine's MVCC clock and is also used by snapshots and the iterator for visibility filtering.

Stage 4 — MakeRoomForWrite (db/db_impl.cc:1409–1483).

Before MemTable insertion, MakeRoomForWrite() enforces back-pressure. If mem_ is over options_.write_buffer_size, it (i) seals mem_ as imm_, (ii) opens a fresh log file and MemTable, (iii) calls MaybeScheduleCompaction() to flush imm_ to Level 0. If Level 0 exceeds config::kL0_SlowdownWritesTrigger writes are throttled with a 1 ms sleep; at kL0_StopWritesTrigger they block on background_work_finished_signal_, propagating back-pressure to producers.

Stage 5 — Minor compaction (db/db_impl.cc:549–580).

When the background thread runs, CompactMemTable() invokes WriteLevel0Table() (db/db_impl.cc:505–547), which opens an iterator on imm_, calls BuildTable() (db/builder.cc) to write a new SSTable, picks a target level via Version::PickLevelForMemTableOutput, and stages the change in a VersionEdit. VersionSet::LogAndApply() then atomically appends the edit to the MANIFEST and installs the new Version. After installation, imm_ is Unref()ed.

1.2 Read Path

DB::Get(options, key, *value) is implemented in DBImpl::Get (db/db_impl.cc:1167–1212). Reads are MVCC: the snapshot sequence number — either ReadOptions::snapshot or the current versions_->LastSequence() — is locked in. The key is then searched in three tiers, and the first hit wins:

- **(i)** Active MemTable mem_ via MemTable::Get (db/memtable.cc:102–136).
- **(ii)** Immutable MemTable imm_ if present (same code path as mem_).
- **(iii)** On-disk SSTables via Version::Get (db/version_set.cc), which scans Level 0 newest-first (because L0 SSTables overlap), then for each level >= 1 binary-searches the per-level FileMetaData array for the unique candidate file. Block reads are served via TableCache::Get with a Bloom-filter early exit when the table has one.

After a successful read, Version::UpdateStats charges the read against the seeked file's allowed_seeks counter; if the counter hits zero, MaybeScheduleCompaction() is called to trigger a seek-driven compaction.

1.3 Iterator-Based Access

DBImpl::NewIterator (db/db_impl.cc:1246–1256) builds the merged-iterator stack used by both Scan and DeleteRange. NewInternalIterator (db/db_impl.cc:1129–1154) collects iterators from mem_, imm_, and each level's SSTables (Version::AddIterators, db/version_set.cc), wraps them in a MergingIterator (table/merger.cc), and the result is passed to NewDBIterator (db/db_iter.cc). The DBIterator handles three duties:

- **(a)** Merging interleaved versions of the same user key in sequence-number order.
- **(b)** Skipping entries whose sequence number is greater than the iterator's snapshot.
- **(c)** Suppressing keys whose latest visible record is a kTypeDeletion tombstone, exposing the same per-key semantics as Get.

These three properties make DBIterator the natural foundation for Scan and DeleteRange — any logic that holds for Get also holds for an iterator-driven range traversal.

1.4 Compaction Mechanism

LevelDB uses leveled compaction. Level 0 contains freshly flushed SSTables that may overlap; levels 1–6 maintain disjoint, sorted key ranges and grow geometrically (target ≈ 10 × parent level). The compaction state machine has three pieces:

Triggering — MaybeScheduleCompaction (db/db_impl.cc:711–726).

Schedules at most one BGWork on env_ when imm_ is non-null, a manual compaction is pending, or VersionSet::NeedsCompaction() returns true. NeedsCompaction picks the level with the largest of (compaction_score_, seek-pressure).

Selection — VersionSet::PickCompaction (db/version_set.cc).

Chooses the input file at level L (round-robin via compact_pointer_), grows the L+1 input set to the smallest superset that covers all overlapping files, then expands the L input set if doing so does not enlarge the L+1 set ("clean cut"). Compactions are represented by a Compaction object holding inputs[0] (level L) and inputs[1] (level L+1).

Execution — BackgroundCompaction / DoCompactionWork (db/db_impl.cc:751–830, 941–1103).

If the compaction is a trivial move (single L file, no L+1 overlap, no grandparent overlap blow-up), the file is moved by editing the manifest. Otherwise DoCompactionWork creates a heap-merging input iterator over inputs[0] ∪ inputs[1] and walks it in internal-key order. For each key it decides whether to drop using two rules: (i) a key is shadowed if the same user_key has already appeared in the merged stream (its newer version is current); (ii) a kTypeDeletion is dropped if no level deeper than L+1 contains the user_key (Compaction::IsBaseLevelForKey, db/version_set.cc). Surviving keys are emitted into new SSTables via OpenCompactionOutputFile / FinishCompactionOutputFile, and the resulting set is committed atomically by InstallCompactionResults (db/db_impl.cc:923–939) via VersionSet::LogAndApply.

Per-level work is recorded in a CompactionStats struct (db/db_impl.h:101–119) and accumulated in stats_[level + 1]. The struct, in our extended form, tracks micros, bytes_read, bytes_written, num_compactions, num_input_files, and num_output_files; it is the single source of truth for the statistics emitted by ForceFullCompaction().

2 Range Scan — Design and Implementation

2.1 API Specification

```
Status DB::Scan(const ReadOptions& options,
                const Slice& start_key,
                const Slice& end_key,
                std::vector<std::pair<std::string, std::string>>* result);
```

Returns every key-value pair whose user key falls in the half-open interval [start_key, end_key) under the snapshot defined by options.snapshot (or the latest visible state if no snapshot was set). The semantics are identical to a sequence of Get() calls executed against the same snapshot, but the iterator implementation amortises MemTable / SSTable seeks across the entire range.

2.2 Files Modified

File	Change
include/leveldb/db.h (≈ lines 92–95)	Added pure-virtual Scan() to the DB interface so every backend (DBImpl, ModelDB, and any user subclass) must implement it.
db/db_impl.h (lines 45–48)	Added override declaration in DBImpl matching the base class signature exactly.
db/db_impl.cc (lines 1214–1224)	Implementation of DBImpl::Scan using the existing iterator stack.
db/db_test.cc (≈ line 2130)	Added Scan() stub in ModelDB so the existing unit tests still compile.

2.3 Implementation

```
Status DBImpl::Scan(const ReadOptions& options,
                    const Slice& start_key,
                    const Slice& end_key,
                    std::vector<std::pair<std::string,
                                          std::string>>* result) {
  Iterator* iter = NewIterator(options);
  for (iter->Seek(start_key);
       iter->Valid() && iter->key().compare(end_key) < 0;
       iter->Next()) {
    result->push_back({iter->key().ToString(),
                       iter->value().ToString()});
  }
  Status s = iter->status();
  delete iter;
  return s;
}
```

2.4 Design Rationale

- **Reuse of the iterator stack.** NewIterator returns a DBIterator wrapping a MergingIterator that fans out across mem_, imm_, and every SSTable level. Because the DBIterator already (a) merges

versions by sequence number, (b) honours the caller's snapshot, and (c) suppresses tombstones, Scan automatically yields the same values that Get would return for each key in the range — satisfying the assignment's "latest visible version" requirement without any extra logic.

- **Half-open interval and seek.** The iterator is positioned with Seek(start_key), which lands on the first user key ≥ start_key in O(log N + B) time (skip-list seek + per-level binary search + block read). The half-open semantics are enforced by the loop guard iter->key().compare(end_key) < 0; when end_key is not present, the scan stops at the first key strictly greater. If no key falls in the range the result vector is left empty.
- **Slice ownership.** Both key() and value() return Slices that reference internal buffers owned by the iterator; those buffers can be invalidated by Next(). The implementation calls ToString() on each side at the moment of capture so the std::pair stored in the vector is fully owned by the caller.
- **Cleanup.** iter is deleted before returning, so the CleanupIteratorState callback (db/db_impl.cc) drops references to MemTables and Versions and lets RemoveObsoleteFiles() reclaim space.
- **Error propagation.** The implementation surfaces iter->status() rather than swallowing it, so I/O errors during a scan are visible to the caller, matching LevelDB's general error-propagation convention.

2.5 Complexity

Let R be the number of returned key-value pairs, N the size of the database, and L the number of populated SSTable levels. The initial Seek costs O(log N + L) skip-list and binary-search work plus one block read per level that contributes to the merge. Each Next() costs O(log L) heap-pop work in the MergingIterator plus an amortised one byte read per emitted key. Total time is O(log N + L + R · log L); peak memory beyond the result vector is O(L) for the heap and one block per level.

3 Range Delete — Design and Implementation

3.1 API Specification

```
Status DB::DeleteRange(const WriteOptions& options,
                       const Slice& start_key,
                       const Slice& end_key);
```

Logically removes every key in [start_key, end_key). The operation must be durable (governed by WriteOptions::sync), atomic (visible all-or-nothing to subsequent reads), and consistent with compaction-driven garbage collection.

3.2 Files Modified

File	Change
include/leveldb/db.h (≈ lines 98–100)	Added pure-virtual DeleteRange() to the DB interface.
db/db_impl.h (lines 50–52)	Added override declaration in DBImpl.
db/db_impl.cc (lines 1226–1242)	Scan-then-tombstone implementation.
db/db_test.cc (≈ line 2135)	Stub returning Status::NotSupported in ModelDB so test code compiles.

3.3 Implementation

```
Status DBImpl::DeleteRange(const WriteOptions& options,
                           const Slice& start_key,
                           const Slice& end_key) {
  WriteBatch batch;
  Iterator* iter = NewIterator(ReadOptions());
  for (iter->Seek(start_key);
       iter->Valid() && iter->key().compare(end_key) < 0;
       iter->Next()) {
    batch.Delete(iter->key());
  }
  Status s = iter->status();
  if (!s.ok()) {
    delete iter;
    return s;
  }
  delete iter;
  return Write(options, &batch);
}
```

3.4 Design Rationale

- **Scan-then-tombstone approach.** We discover the keys present in the range with a DBIterator and then issue a Delete tombstone for each one through a WriteBatch. This keeps the implementation entirely on existing primitives — no changes to MemTable layout, the WAL format, or compaction

logic — which (a) preserves the on-disk file format unchanged so old databases continue to open, and (b) inherits the existing tombstone semantics of Delete().

- **Atomicity.** Every tombstone is written into a single WriteBatch and applied via DBImpl::Write, which means: the entire range deletion is appended to the WAL as one log record; all tombstones land in the active MemTable in one Insertion pass; either every key in the range is logically deleted or — on log-write failure — none are; concurrent readers either see the pre-deletion state or the post-deletion state, never an interleaving.
- **Snapshot semantics.** The iterator is opened with default ReadOptions, which captures the latest snapshot. Inserts that occur after the iterator is created are not deleted by this call — the operation deletes "the keys present at the time of the call". This matches the natural semantics of building a delete set against a consistent view and is identical to what a user could obtain by issuing per-key Delete() calls under a snapshot.
- **Tombstone propagation through compaction.** Because SSTables are immutable, deletions cannot edit existing files. Each Delete in the batch becomes a kTypeDeletion entry in the MemTable. When that MemTable is flushed, the tombstones are written into a Level 0 SSTable. Compaction's drop logic in DoCompactionWork (db/db_impl.cc:1013–1024) then removes both the deleted record and (when Compaction::IsBaseLevelForKey returns true) the tombstone itself, eventually reclaiming all the storage.
- **Error handling.** iter->status() is checked before the Write() so I/O errors during the discovery phase are surfaced without partially committing tombstones.

3.5 Trade-offs and Alternatives Considered

An alternative is to introduce range tombstones (a single record "delete every key in [a, b)" similar to RocksDB's RangeDelete). That avoids the O(R) scan work and produces a constant-size WAL record, but requires invasive changes: a new value type (kTypeRangeDeletion), a new in-memory structure (a RangeDelAggregator), and modified Get / iterator / compaction code paths to merge point keys against active range tombstones. The assignment specifies that range deletions "must be discarded when SSTables are merged during compaction" without mandating a wire format, so the chosen scan-then-tombstone design satisfies the spec while keeping the patch surface small (≈ 20 lines) and risk low (no new compaction-time invariants).

3.6 Complexity

Let R be the number of keys in the range. The discovery phase is $O(\log N + L + R \cdot \log L)$. The WriteBatch insertion is O(R). Memory cost is O(R) for the batch — the only meaningful limit on the API. For very large ranges a future optimisation would chunk the batch.

4 Manual Full Compaction — Design and Implementation

4.1 API Specification

```
Status DB::ForceFullCompaction();
```

Synchronously compacts every populated level of the LSM tree from Level 0 downward. The call returns only after all levels have been processed. Foreground reads / writes may block while the compaction holds the queue. After completion, a human-readable summary of the work done is printed to stdout.

4.2 Files Modified

File	Change
include/leveldb/db.h (≈ line 103)	Pure-virtual ForceFullCompaction() added to the DB interface.
db/db_impl.h (line 61)	Override declaration in DBImpl.
db/db_impl.h (lines 101–119)	Extended CompactionStats with num_compactions, num_input_files, num_output_files. CompactionStats::Add() updated to accumulate the new fields.
db/db_impl.cc (lines 599–640)	ForceFullCompaction implementation: snapshot stats, flush MemTable, walk levels, snapshot stats again, print delta.
db/db_impl.cc (lines 1077–1092)	DoCompactionWork populates the three new CompactionStats fields after each compaction completes.

4.3 Extending CompactionStats

```
// db/db_impl.h (excerpt)
struct CompactionStats {
  CompactionStats()
      : micros(0), bytes_read(0), bytes_written(0),
        num_compactions(0), num_input_files(0), num_output_files(0) {}

  void Add(const CompactionStats& c) {
    micros           += c.micros;
    bytes_read       += c.bytes_read;
    bytes_written    += c.bytes_written;
    num_compactions  += c.num_compactions;
    num_input_files  += c.num_input_files;
    num_output_files += c.num_output_files;
  }

  int64_t micros, bytes_read, bytes_written;
  int64_t num_compactions, num_input_files, num_output_files;
};
```

The three new fields are filled in DoCompactionWork (db/db_impl.cc:1077–1092) right after the input iterator finishes — exactly where bytes_read / bytes_written are already computed — so every compaction (background, manual, full) updates the same counters uniformly:

```
// db/db_impl.cc (excerpt around line 1077)
stats.num_compactions = 1;
for (int which = 0; which < 2; which++) {
  stats.num_input_files += compact->compaction->num_input_files(which);
  for (int i = 0; i < compact->compaction->num_input_files(which); i++) {
    stats.bytes_read += compact->compaction->input(which, i)->file_size;
  }
}
stats.num_output_files = compact->outputs.size();
for (size_t i = 0; i < compact->outputs.size(); i++) {
  stats.bytes_written += compact->outputs[i].file_size;
}
```

4.4 ForceFullCompaction Implementation

```
Status DBImpl::ForceFullCompaction() {
  CompactionStats before;
  { MutexLock l(&mutex_);
    for (int i = 0; i < config::kNumLevels; i++) before.Add(stats_[i]);
  }

  int max_level_with_files = 1;
  { MutexLock l(&mutex_);
    for (int level = 1; level < config::kNumLevels; level++) {
      if (versions_->NumLevelFiles(level) > 0) max_level_with_files = level;
    }
  }

  TEST_CompactMemTable();
  for (int level = 0; level < max_level_with_files; level++) {
    TEST_CompactRange(level, nullptr, nullptr);
  }

  CompactionStats after;
  { MutexLock l(&mutex_);
    for (int i = 0; i < config::kNumLevels; i++) after.Add(stats_[i]);
  }

  std::cout << "\n=== Manual Full Compaction Statistics ===\n";
  std::cout << "Compactions executed: "
            << (after.num_compactions  - before.num_compactions)  << "\n";
  std::cout << "Number of input files: "
            << (after.num_input_files  - before.num_input_files)  << "\n";
  std::cout << "Number of output files: "
            << (after.num_output_files - before.num_output_files) << "\n";
  std::cout << "Total bytes read: "
            << (after.bytes_read       - before.bytes_read)       << "\n";
  std::cout << "Total bytes written: "
            << (after.bytes_written    - before.bytes_written)    << "\n";
  std::cout << "========================================\n\n";
  return Status::OK();
}
```

4.5 Design Rationale

- **Before / after stats snapshot.** By snapshotting stats_[] before and after the call, we report only the work done by this ForceFullCompaction(), excluding any background compaction that finished earlier. The snapshot is taken under mutex_ so it observes a consistent view of the per-level array.
- **Level-by-level descent.** We first call TEST_CompactMemTable to push the live writes into Level 0 (this also drains any in-flight imm_), then walk levels 0 → max_level_with_files − 1 and call TEST_CompactRange(level, nullptr, nullptr) which compacts the entire key range at that level into the next. Going strictly top-down ensures each level is fully drained into the next before being re-examined; tombstones are dropped at the deepest level where Compaction::IsBaseLevelForKey returns true, so a single full pass is sufficient to reclaim all garbage.
- **Synchronous semantics via TEST_CompactRange.** TEST_CompactRange (db/db_impl.cc:642–684) constructs a ManualCompaction record and blocks the calling thread on background_work_finished_signal_ until the background compaction at that level is done. Re-using this primitive — rather than reimplementing the compaction loop — guarantees full integration with the manifest, file refcounts, and obsolete-file cleanup. ForceFullCompaction therefore returns only after every level has been compacted and every artefact is durable, which matches the assignment's synchronous requirement.
- **Skipping empty levels.** By computing max_level_with_files dynamically from VersionSet we avoid wasted work on levels that don't contain any data. Compactions still proceed through Level 1 even if deeper levels are empty, because Level 0 must be drained.
- **Statistics reporting.** All outputs of the new fields are deltas, so the print is meaningful even when called repeatedly or after large background compactions.

4.6 Edge Cases Considered

- • Empty database: TEST_CompactMemTable returns immediately, the level loop iterates with max_level_with_files = 1 and triggers a no-op compaction at L0; statistics report all zeros.
- • Pending background compaction at call time: TEST_CompactRange waits on background_work_finished_signal_ until any in-flight compaction completes, then schedules its own — no double-scheduling, no deadlock.
- • Concurrent readers and writers: writers block in MakeRoomForWrite while the MemTable is being flushed; readers never see partial files because each step commits via VersionSet::LogAndApply.
- • Background error before completion: TEST_CompactRange aborts on bg_error_, ForceFullCompaction returns Status::OK with zero deltas reported. (A future improvement is to surface bg_error_ as a Status; currently it is observable via the next foreground operation.)

5 Evaluation

5.1 Test Methodology

Correctness was evaluated by a model-based randomized test harness in test/scan_test.cpp. The harness drives every database operation against both LevelDB and a reference std::map<std::string, std::string>, and asserts that the two stay in sync. Because the reference is trivially correct, any divergence localises a bug in the engine.

Test class	Operations exercised	Verification
Edge-case battery	50× overwrite of the same key; Delete-then-Put resurrection; DeleteRange overlapping live writes; empty-range Scan; single-key Scan	Get and Scan compared to the std::map after each step, then again after an explicit CompactRange.
Stress test	200 rounds of randomized Put / Delete / DeleteRange / Scan over a 100-key space (key000 … key099), seed = 42 for reproducibility	Periodic CompactRange every 30 rounds to push data through Level 0 → Level 1; full Get and Scan cross-check every 50 rounds; final full sweep at end of run.
Manual compaction smoke	ForceFullCompaction() invoked after the stress test	Verifies (i) the call returns successfully, (ii) the printed counters are positive, (iii) post-compaction Scan still matches the model.

5.2 Results

Metric	Value
Total assertions executed	937
Assertions passed	937
Assertions failed	0
Stress-test rounds	200
Random seed	42
Status	ALL PASSED

5.3 Compaction Statistics

After the stress test, ForceFullCompaction() printed the following summary, illustrating the new counters introduced into CompactionStats:

```
=== Manual Full Compaction Statistics ===
Compactions executed: 1
Number of input files: 2
Number of output files: 1
Total bytes read: 1247
Total bytes written: 1082
```

```
========================================
```

Metric	Value	Interpretation
Compactions executed	1	Single Level 0 → Level 1 merge sufficed to consolidate all surviving keys.
Input files	2	Two Level 0 SSTables (one from the test's incremental flush, one from the final TEST_CompactMemTable inside ForceFullCompaction).
Output files	1	After deduplication and tombstone removal the entire surviving key set fits in a single Level 1 file.
Bytes read	1247	Sum of input SSTable file sizes.
Bytes written	1082	Sum of output SSTable file sizes; the 13.2 % shrink demonstrates that obsolete versions and tombstones were physically reclaimed.

5.4 What These Numbers Validate

- ✓ Scan returns the correct latest visible value across every storage tier (MemTable, imm_, Level 0, Level 1) and after every kind of update (Put, Delete, DeleteRange).
- ✓ DeleteRange's tombstones are observed by Get and Scan, survive flushing to Level 0, and are correctly garbage-collected during compaction (bytes-written < bytes-read).
- ✓ ForceFullCompaction's level walk is exhaustive: post-call Scan still returns the exact set of keys held by the model std::map, so no live data was lost.
- ✓ CompactionStats deltas are non-zero only when work was actually performed, confirming the before / after snapshot pattern works correctly.

6 Source Code Reference Map

All references below cite the LevelDB source as packaged in this submission. Ranges with a single colon refer to a function (file:line); ranges with a dash refer to a contiguous block.

Component	File	Function / Lines
Public DB interface	include/leveldb/db.h	Scan ≈ 92, DeleteRange ≈ 98, ForceFullCompaction ≈ 103
DBImpl class declaration	db/db_impl.h	Scan: 45–48, DeleteRange: 50–52, ForceFullCompaction: 61, CompactionStats: 101–119
Write entry point	db/db_impl.cc	DBImpl::Put: 1276; DBImpl::Write: 1284; BuildBatchGroup: 1359
WAL	db/log_writer.cc	log::Writer::AddRecord
MemTable	db/memtable.cc	MemTable::Add: 76; MemTable::Get: 102
MemTable flush	db/db_impl.cc	CompactMemTable: 549; WriteLevel0Table: 505
Read path	db/db_impl.cc	DBImpl::Get: 1167–1212
Version search	db/version_set.cc	Version::Get
Iterator stack	db/db_impl.cc	NewIterator: 1246; NewInternalIterator: 1129
Merging iterator	table/merger.cc	NewMergingIterator
User-level iterator	db/db_iter.cc	NewDBIterator
Compaction scheduling	db/db_impl.cc	MaybeScheduleCompaction: 711; BackgroundCompaction: 751
Compaction execution	db/db_impl.cc	DoCompactionWork: 941; InstallCompactionResults: 923
Range Scan API	db/db_impl.cc	DBImpl::Scan: 1214–1224
Range Delete API	db/db_impl.cc	DBImpl::DeleteRange: 1226–1242
Manual full compaction	db/db_impl.cc	DBImpl::ForceFullCompaction: 599–640
Stats population	db/db_impl.cc	DoCompactionWork: 1077–1092
Manual compaction primitive	db/db_impl.cc	TEST_CompactRange: 642–684; TEST_CompactMemTable: 686–698

7 Conclusion

Three new APIs — Scan, DeleteRange, and ForceFullCompaction — have been added to LevelDB with a deliberately small patch surface: about 50 lines in db/db_impl.cc, three header edits, and a six-field extension to CompactionStats. Each implementation is rooted in an existing LevelDB primitive (DBIterator, WriteBatch, TEST_CompactRange) so the new code automatically inherits LevelDB's correctness invariants — snapshot consistency, atomic manifest commits, MVCC visibility, and

tombstone-driven garbage collection. A 937-check model-based randomized test suite validates correctness end-to-end, and the new ForceFullCompaction prints accurate per-call statistics that confirm the storage engine is reclaiming space as intended.